AN0290
应用笔记

AT32F405 高速USB磁轴键盘应用笔记

前言

磁轴键盘（Magnetic Switch Keyboard）是一种采用 磁性感应技术作为按键触发方式的键盘产品。相比传统机械轴或薄膜键盘，磁轴键盘具有以下优势：

- 非接触式感应：通过霍尔传感器检测磁体位置变化来判断按键状态，不依赖金属接点，降低磨损和故障风险。
- 高耐用性：按键不直接接触，寿命可达数千万次以上，适合长时间使用。
- 高速响应：磁轴传感器提供快速、稳定的信号变化，结合 MCU 高速扫描，实现低延迟按键响应。
- 可调节特性：按键行程、触发点和回弹点可通过硬件或软件进行调节，提供定制化使用体验

磁轴键盘广泛应用于 电竞键盘、高端输入设备、专业控制面板等场景，通过 MCU 系统可以实现按键扫描、USB HID 通讯及 LED 背光控制的高度集成。

这是一个基于AT32 MCU 的高速USB磁轴键盘设计Demo说明文档，具备 8kHz 上报率、24kHz 扫描率，并支持按键灵敏度调节等功能。

支持型号列表：

支持型号	AT32F405

目录

1 概述 5

1.1 硬件环境 5

1.2 软件环境 6

2 软件实现 8

2.1 键盘扫描 8

2.1.1 扫描流程简介 8

2.1.2 代码流程 9

2.2 按键状态 10

2.2.1 按键状态机说明 10

2.3 USB HID KEY 上报 12

2.4 USB Suspend 功耗管理机制 13

2.5 RGB LED 控制 13

3 版本历史 15

表目录

表 1. 按键变量表 11
表 2. 文档版本历史..... 15

图目录

图 1 整体架构 5
图 2 AT32-Magnetic-Gamings-Keyboard 实物图 6
图 3 IAP 上位机升级 7
图 4 模拟复用器连接示意图 8
图 5 按键扫描流程图 9
图 6 按键状态更新图 10

1 概述

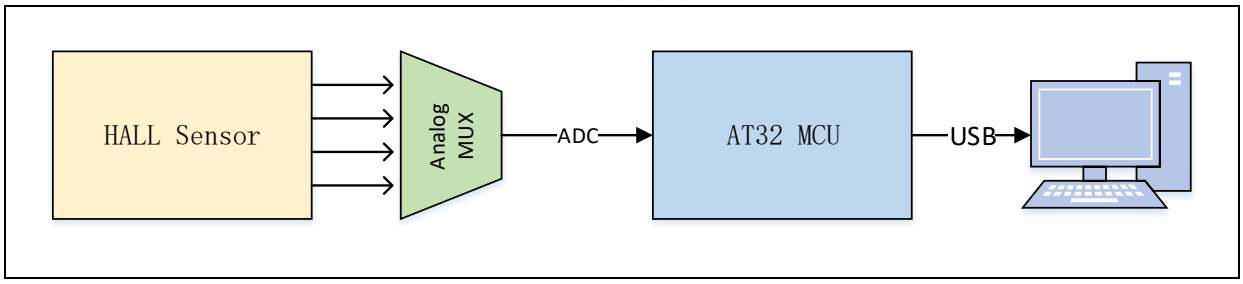
本应用说明文件介绍一款基于AT32 MCU 高速USB磁轴键盘的设计Demo。

内容包括系统架构，硬件设计，软件架构（IAP+APP）、按键扫描（ADC+DMA+模拟复用器）、USB通信，以及WS2812 RGB LED的控制实现，供开发者作为磁轴键盘产品的开发参考。

主要特性：

- 8kHz上报率
- 按键扫描率达到16kHz
- 磁轴霍尔感测按键输入
- 实现68键键位设计
- ADC+DMA+模拟复用器高效率按键扫描
- 可选择Kalman 1阶/Kalman 2阶/ EMA（指数移动平均）算法进行数据滤波
- 支持触发灵敏度调整
- WS2812 RGB LED显示
- USB High-Speed 接口
- IAP+APP架构，支持在线升级

图 1 整体架构



1.1 硬件环境

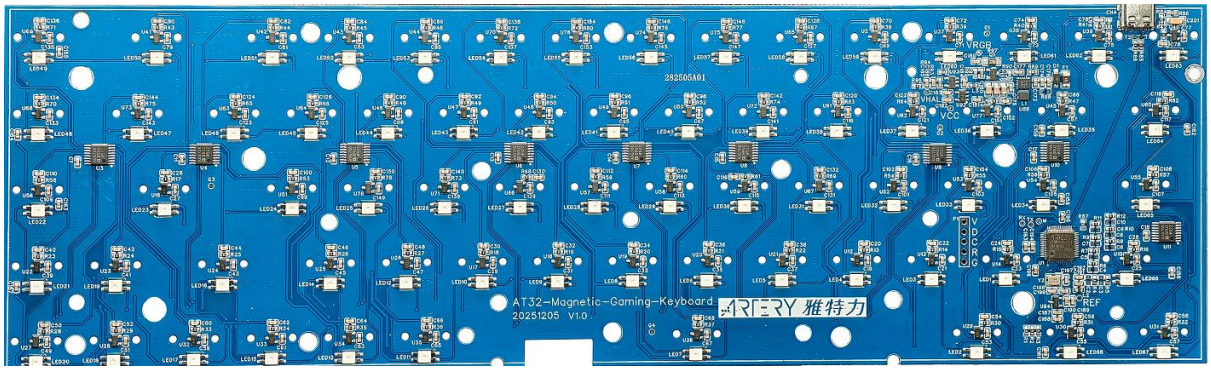
本 Demo 基于 AT32-Magnetic-Gamings-Keyboard 开发板，该开发板以 AT32 MCU 实现磁轴键盘完整的功能，主要包含以下器件：

- AT32 MCU 主控芯片
- 霍尔传感器阵列（用于磁轴按键感测）
- 模拟复用器（8:1，用于扩展 ADC 输入通道）
- WS2812 可寻址 RGB LED
- USB High-Speed 接口

实际电路图可参考文件《SCH_AT32-Magnetic-Gaming-Keyboard.Vxx.pdf》

如下是开发板实物图：

图 2 AT32-Magnetic-Gamings-Keyboard 实物图



1.2 软件环境

本应用笔记使用 IAP+APP 架构，工程路径如下：

- **IAP bootloder**

  路径：utilities\bootloader\mdk_v5

  功能：固件升级与程序跳转

  Flash 地址空间:0x08000000~0x080006000（24 KB）

- **Keyboard APP**

  路径：utilities\keyboard\mdk_v5

  功能：键盘扫描，USB 通讯及 RGB LED 控制显示

  Flash 地址空间: 0x080006000~0x08040000（232 KB）

Keybaord APP 工程主要结构：

keyboard/

```
├── src/
│   ├── main.c // 系统入口
│   ├── keyboard.c // 键盘扫描 + 状态机核心
│   ├── keyboard_adc.c // 键盘 ADC
│   ├── keyboard_filter.c // 键盘按键滤波算法
│   ├── keyboard_power.c // 键盘挂起状态功耗处理
│   ├── keyboard_usb.c // 键盘 USB 接口
│   ├── keymap.c // 物理键索引与 HID 键值映射
│   ├── keyboard_class.c // USB HID 键盘类相关处理
│   ├── keyboard_desc.c // HID 描述符
│   ├── hid_iap_user.c // HID IAP 相关处理
│   └── rgb_led.c // WS2812 LED 驱动
```

使用步骤:

1. 先烧录 Bootloader 程序
2. 再烧录 Keyboard APP 程序
3. 上电后由 Bootloader 跳转至 APP 执行

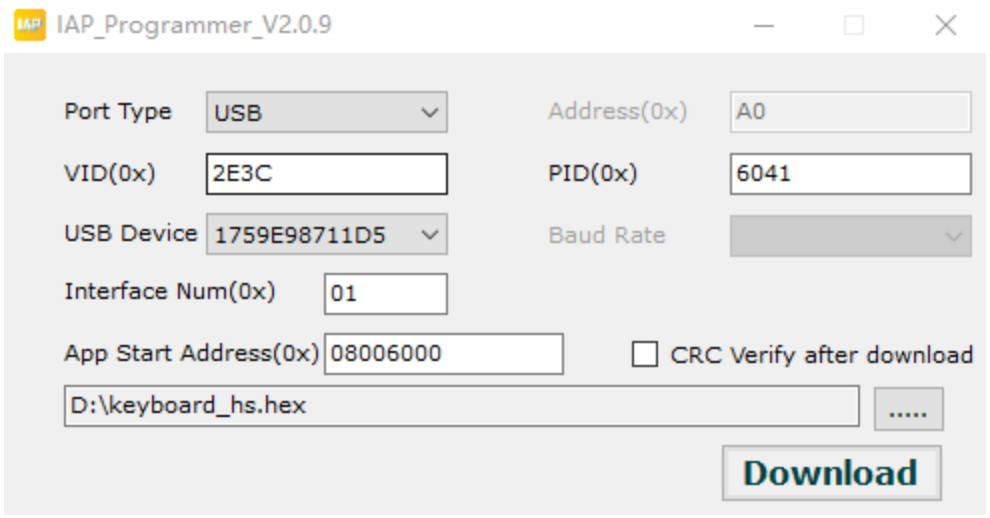
在线升级可通过配套的上位机工具完成，Tool/IAP_Programmer

IAP 上位机使用方法:

- 打开 IAP Programmer
- Port type 选择 USB
- Interface Num 选择 0x01
- PID 输入 0x6041
- 选择对应的 USB Device，以及对应的 hex 文件
- 点击 Download 进行下载

如下图：

图 3 IAP 上位机升级



Note: *本应用笔记的项目基于 keil 5 而建立，若用户需要在其他编译环境上使用，请参考 AT32Fxx_Firmware_Library_V2.x.x\project\at_start_f4xx\templates 中各种编译环境（例如 IAR6/7,keil 4/5）进行简单修改即可。*

2 软件实现

本章节主要介绍磁轴键盘 Demo 的系统实现方式，包括键盘扫描、按键状态判定、RGB LED 控制以及 USB HID 通讯。

系统上电后软件执行流程：

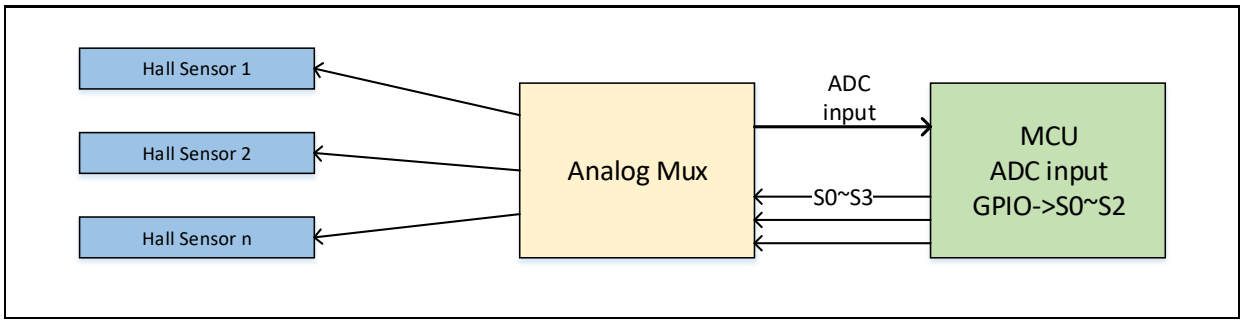
```
main()
├── system_clock_config() //时钟初始化
├── wk_timebase_init() //延时函数初始化
├── usbd_init()          //USB 初始化
├── rgb_led_init()       //ws2812 RGB LED 初始化
├── keyboard_init()      //按键扫描（ADC,DMA）初始化与校准
└── while(1)
    └── keyboard_task() //按键扫描+算法处理+上报
    └── rgb_led_task()  //RGB LED 处理
```

2.1 键盘扫描

磁轴键盘采用 **ADC + DMA + 模拟复用器** 的方式进行高效按键扫描。每个按键对应一个霍尔传感器，其输出电压与磁体位置（即按键行程）成线性或准线性关系，因此可以获取连续、可量化的位移信息。

由于键盘按键数量较多，而 MCU 可用的 ADC 输入通道数量有限，在硬件上引入模拟复用器（8:1） 对霍尔传感器信号进行通道扩展。多个按键信号被接入模拟复用器的输入端，通过控制复用器的选择信号，将指定通道的模拟电压输出至 MCU 的 ADC 输入端，从而实现对多按键的轮询采样。

图 4 模拟复用器连接示意图



在软件实现上，系统按照既定的扫描顺序依次切换模拟复用器通道，并在每次通道切换稳定后触发 ADC 转换。ADC 采样结果通过 DMA 自动搬运至内存缓冲区，使用双缓冲结构，ADC 转换和按键解析可以同时进行，来提高效率。

2.1.1 扫描流程简介

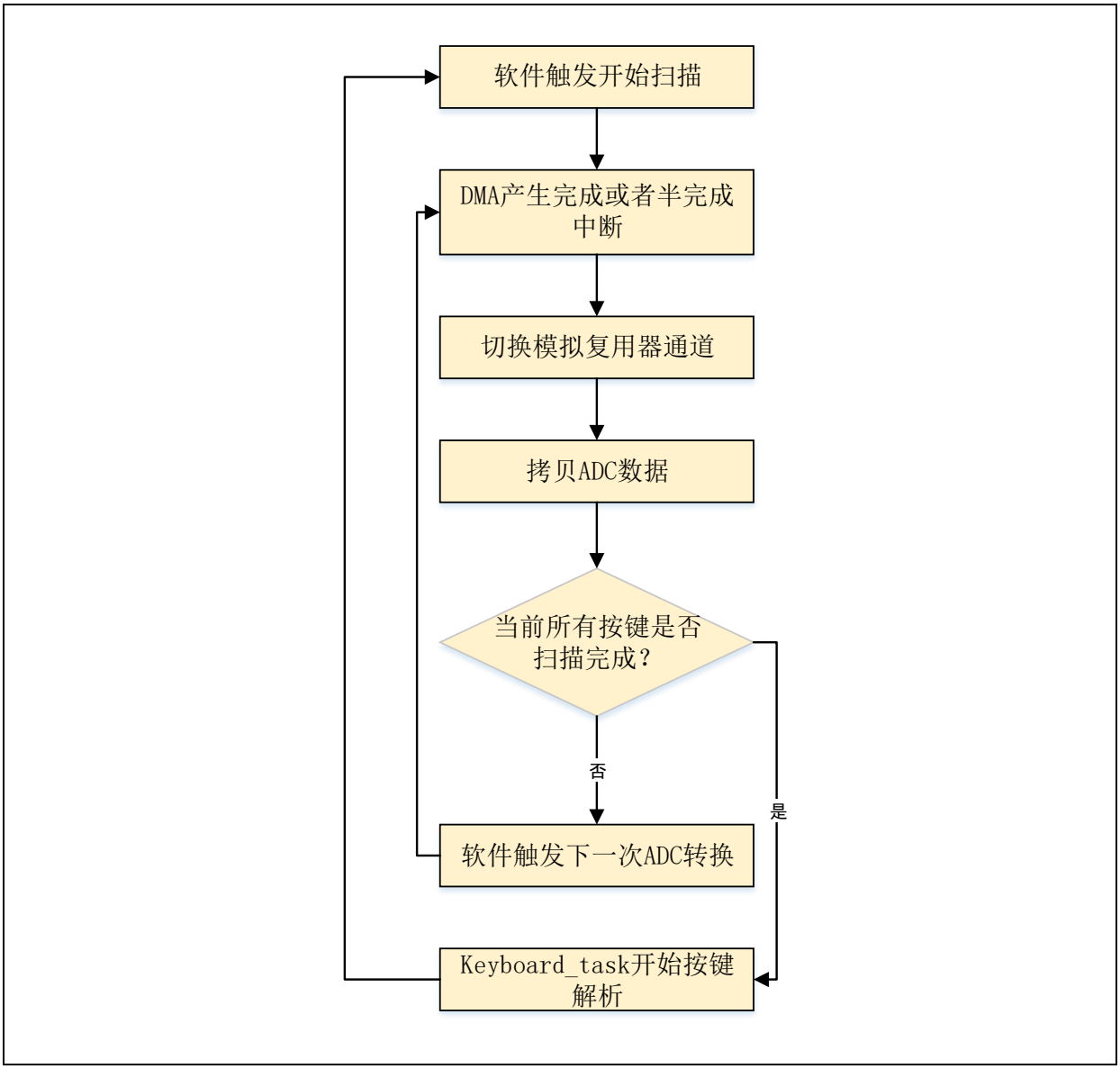
全键盘总共有 68 个按键，每个按键会对应一个 ADC 的采样值。每个 ADC 通道都通过一个（8:1）模拟复用器扩展接到外面的霍尔感应器。在进行全键盘扫描的时候，ADC 每次转换完成一个序列之后，都需要去切换模拟复用器接到下一个霍尔感应器进行采样。

采样步骤如下：

- 软件触发 ADC 开始一次按键扫描开始
- DMA 产生半完成或完成中断

- MCU 控制 GPIO 切换模拟复用器通道
- 在通道稳定后启动 ADC 下一次开始转换
- 如果已经完成一次全键盘扫描，等待 CPU 开启下一次扫描

图 5 按键扫描流程图



2.1.2 代码流程

如下列出扫描过程中代码执行流程：

adc_ordinary_software_trigger_enable() //软件触发 ADC 序列模式开始转换

```
├── KEY_ADC_DMA_IRQ_HANDLER() //一次序列转换完成中断处理
    └── key_matrix_cs_switch() //切换下一次复用器通道
    └── adc_ordinary_software_trigger_enable() //触发下一次 ADC 转换
    └── copy_data //拷贝当前完成的数据
└── keyboard_task //按键状态解析
```

2.2 按键状态

由于磁轴按键输出为模拟信号，且在实际应用中会受到电源纹波、环境噪声及器件离散性的影响，必须对 ADC 采样数据进行合理处理，才能可靠判断按键状态并进一步生成有效的键值信息。

本 Demo 提供多种滤波算法选项，包括指数移动平均（EMA，Exponential Moving Average）、Kalman 1 以及 Kalman 2，用于对原始 ADC 采样数据进行滤波处理，以在系统响应速度与信号稳定性之间实现不同侧重的平衡。

Kalman 1 采用一维 Kalman 滤波算法，通过固定参数对系统噪声与测量噪声进行建模，能够根据估计误差动态调整滤波增益，在保持较高稳定性的同时显著降低滤波延迟，适用于游戏键盘及 Rapid Trigger 等对实时性要求较高的应用。

Kalman 2 进阶滤波模式，在 Kalman 1 的基础上引入更高精度或自适应策略，可根据按键运动状态动态调整滤波特性，在快速响应与静态稳定性之间实现更优平衡，提供更低延迟与更佳的跟手性，适用于对性能与手感要求最高的高阶类比键盘应用。

系统基于按键行程与相对位移变化对按键状态进行判定，并通过可配置的触发位移阈值实现按键灵敏度调节，从而满足不同应用场景下对触发精度与操作手感的需求。

2.2.1 按键状态机说明

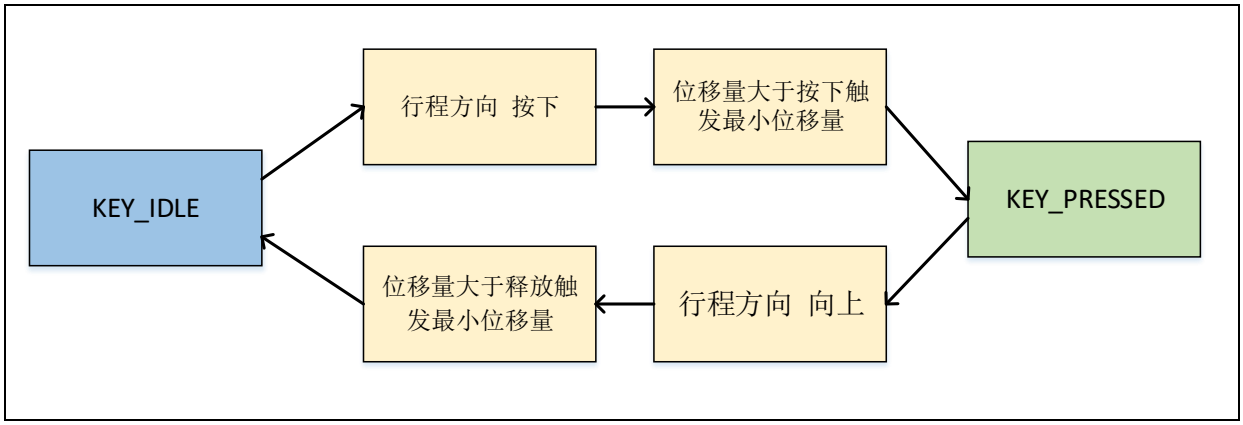
磁轴键盘基于霍尔传感器获取按键的连续位移信息，与传统机械键盘的“开 / 关”触点不同，其按键行为本质上是一个连续模拟量变化过程。

若仍采用固定触发行程与固定释放行程的判定方式，将会带来以下问题：

- 快速连击时释放不及时
- 半行程操作响应迟滞
- 无法充分发挥磁轴的连续行程优势

因此，本设计引入一个简单的 Rapid Trigger（RT）按键状态机，通过分析按键行程方向与相对位移变化，实现动态触发与释放，如下是按键状态机更新图：

图 6 按键状态更新图



2.2.1.1 状态机设计

状态机不依赖绝对 ADC 阈值，而是基于以下三点：

- 行程方向判断（dv）
- 单次行程极值记录（adc_max_value/ adc_min_value）

- 相对位移变化量（rt_press_delta/ rt_release_delta）

核心判定原则可以概括为一句话：
当按键在某一方向上形成了“足够明确的位移趋势”，即触发状态切换。可通过调整相对位移变化量（rt_press_delta/ rt_release_delta）来实现按键灵敏度的调整。

2.2.1.2 关键变量说明

代码里面的关键变量说明，用于判断按键的状态：

表 1. 按键变量表

变量	说明
adc_filter_value	当前按键 ADC EMA 后位移值
dv	位移变化量，用于判断行程方向
DV_DEADZONE	行程方向死区，抑制微小抖动
adc_max_value	本次过程中的最大位移值
adc_min_value	本次过程中的最小位移值
rt_press_delta	按下触发所需的最小相对位移
rt_release_delta	释放触发所需的最小相对位移

2.2.1.3 状态逻辑

状态机包含两个基本状态：

- **RT_KEY_IDLE**
  按键处于释放或待机状态，系统监测按下趋势
- **RT_KEY_PRESSED**
  按键已被判定为按下状态，系统监测回弹趋势

1. 按下判断逻辑

行程最大值记录

```
if (dv > DV_DEADZONE  && value > k->adc_max_value)
    k->adc_max_value = value;
```

当检测到按键位移呈现明显的“向下按压”趋势时，持续更新本次行程中的最大位移值，为后续相对位移判断提供参考基准。

按下触发条件

```
if (dv <= DV_DEADZONE  &&
    k->adc_max_value - value > k->rt_press_delta)
```

按下触发需要同时满足：

- 当前位移变化已趋于稳定（进入方向死区）
- 相对于本次最大位移，回落幅度超过 rt_press_delta

该设计可有效避免：

- 微小抖动导致的误触发
- 行程尚未形成明确趋势时的错误判定

一旦条件满足，按键状态切换为 RT_KEY_PRESSED，并生成对应的 HID 按键按下事件。

2. 释放判断逻辑

行程最小值记录

```
if (dv < DV_DEADZONE  && value < k->adc_min_value)
```

```
        k->adc_min_value = value;
```

当检测到按键位移呈现“向上回弹”趋势时，记录回弹过程中的最小位移值。

释放触发条件

```
if (dv >= DV_DEADZONE  &&
        (value - k->adc_min_value > k->rt_release_delta))
```

释放触发条件为：

- 位移趋势重新进入稳定或向下方向
- 相对于最小位移的回升幅度超过 rt_release_delta 该设计可有效避免：

该机制允许按键在尚未完全回弹到初始位置时提前释放，显著提升快速连击与回拉操作的响应速度。

3. 参数学习（auto learn）

在启用 SUPPORT_AUTO_LEARN_RT 时，系统可在一次完整按压-释放行程结束后，对 RT 参数进行自学习更新，用于适配不同轴体或磁体差异。

为保证学习结果稳定，建议仅在行程幅度达到预设最小值时执行更新。

2.3 USB HID KEY 上报

磁轴键盘系统通过 USB High-Speed HID（Human Interface Device）协议与主机通信，实现标准键盘输入功能，无需额外驱动程序即可使用。

2.3.1.1 KEY 上报接口说明

USB HID 键盘上报通过统一接口函数发送：

```
uint8_t hid_keyboard_report(uint8_t *data, uint8_t len)
{
   if(p_key->is_suspend) {
      clear_keyboard_suspend();
      usbd_remote_wakeup(&otg_core_struct.dev);
      return 0;
   } else {
      return (uint8_t)usb_keyboard_class_send_report(&otg_core_struct.dev, (uint8_t *)data, 8);
   }
}
```

该接口负责处理以下两类场景：

1. 正常键盘数据上报
2. USB 挂起（Suspend）状态下的远程唤醒(Remote Wakeup)

- 正常数据上报

  usb_keyboard_class_send_report(&otg_core_struct.dev, (uint8_t *)data, 8);

  报告长度固定为 8 字节
- Suspend 远程唤醒

  当键盘处于 Suspend 状态且检测到按键事件时：

  1. 清除本地 Suspend 标志
  2. 通过 usbd_remote_wakeup()向主机发送远程唤醒信号

2.3.1.2 键值管理

键盘采用 6 key Rollover 结构，一次上报同时支持 6 个普通按键。

添加按键：

```
static void keyboard_add_key(uint8_t keycode)
```

此函数按顺序将键值写入 keycode[]中，当超过 6 键时自动回绕。

清除按键：

```
static void keyboard_clear_key(uint8_t keycode)
```

遍历 6 个键位，匹配到对应键值后清 0

2.4 USB Suspend 功耗管理机制

当 USB 总线进入 Suspend 状态时，系统将通过降低 MCU 主频、关闭 RGB 的显示来降低系统的整体功耗。

- **降低 MCU 主频**

  当设备进入 suspend 之后， 通过调试系统主频到 48MHz 来降低功耗。

```
if(p_key->is_lowpower == FALSE && usbd_is_keyboard_suspend()== TRUE) {
   /* enter low power */
   suspend_enter_lower_power();
}
```

- **关闭 RGB 显示**

  进入 suspend 之后，会关闭 RGB 来降低功耗

```
if(usbd_is_keyboard_suspend()) {
      if(rgb_led_suspend == 0) {
            rgb_led_suspend = 1;
            LED_RGB_OFF();
      }
      return;
}
```

2.5 RGB LED 控制

本磁轴键盘 Demo 集成了 WS2812 可寻址 RGB LED，demo 实现简单的呼吸灯和跑马灯效果。WS2812 通过单线串行方式传输数据，对时序要求较高，因此在软件实现上需保证信号的精确性与稳定性。

Demo 中提供了两种典型实现方式：

控制方式	宏定义	特点
SPI+DMA	WS2812_CONTROL_USE_SPI	利用 SPI 模拟 WS2812 波形，并通过 DMA 自动发送数据
TMR(PWM)+DMA	WS2812_CONTROL_USE_TMR	使用定时器输出 PWM 波形，由 DMA 动态更新比较值

上述方案均可在不占用大量 CPU 资源的情况下，稳定驱动大数量的 RGB LED。

在灯效设计上，Demo 提供了基础的显示模式，包括：

- 呼吸灯效果
- 流水灯效果

开发者可根据需要，将 LED 显示与按键状态、工作模式或主机指令进行联动，实现更加丰富和个性化的灯效表现。

3 版本历史

表 2. 文档版本历史

日期	版本	变更
2026.01.22	2.0.0	初始版本

重要通知 – 请仔细阅读

买方自行负责对本文所述雅特力产品和服务的选择和使用，雅特力概不承担与选择或使用本文所述雅特力产品和服务相关的任何责任。

无论之前是否有过任何形式的表示，本文档不以任何方式对任何知识产权进行任何明示或默示的授权或许可。如果本文档任何部分涉及任何第三方产品或服务，不应被视为雅特力授权使用此类第三方产品或服务，或许可其中的任何知识产权，或者被视为涉及以任何方式使用任何此类第三方产品或服务或其中任何知识产权的保证。

除非在雅特力的销售条款中另有说明，否则，雅特力对雅特力产品的使用和/或销售不做任何明示或默示的保证，包括但不限于有关适销性、适合特定用途（及其依据任何司法管辖区的法律的对应情况），或侵犯任何专利、版权或其他知识产权的默示保证。

雅特力产品并非设计或专门用于下列用途的产品：（A）对安全性有特别要求的应用，例如：生命支持、主动植入设备或对产品功能安全有要求的系统；（B）航空应用；（C）航天应用或航天环境；（D）武器，且/或（E）其他可能导致人身伤害、死亡及财产损害的应用。如果采购商擅自将其用于前述应用，即使采购商向雅特力发出了书面通知，风险及法律责任仍将由采购商单独承担，且采购商应独力负责在前述应用中满足所有法律和法规要求。

经销的雅特力产品如有不同于本文档中提出的声明和/或技术特点的规定，将立即导致雅特力针对本文所述雅特力产品或服务授予的任何保证失效，并且不应以任何形式造成或扩大雅特力的任何责任。

© 2026 雅特力科技 保留所有权利